Pakistan Navy Complaint Management System (CMS)

Document 06: Code Commenting Standard & Audit Guide

Document Identifier: CMS-CODE-V1.2.4
Target Audience: Development Leads, Backend Engineers, Software Quality Auditors
Date: September 2026
Status: Approved for Handover

1. Objective & Code Quality Standards

Clean, transparent, and well-commented source code is essential for the long-term maintainability and security auditing of the **Pakistan Navy Complaint Management System (CMS)**.

This document defines the strict code documentation standards required across all PHP, Blade, JavaScript, and SQL source files in the project. All code merged into the production branch (`deploy`) MUST adhere to **PSR-12** standards and include standardized **DocBlocks**.

2. DocBlock & Commenting Conventions

2.1 File & Class Header DocBlocks

Every class file MUST begin with a descriptive header defining the class purpose, package namespace, and architectural responsibilities:

```
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Models\Complaint;
use Illuminate\Http\Request;
use Illuminate\View\View;

/**
 * Class ComplaintController
 *
 * Handles the core complaint lifecycle within the administrative portal.
 * Orchestrates status transitions, technician dispatching, attachment processing,
 * and integration with the spare parts performa approval workflow.
 *
 * @package App\Http\Controllers\Admin
 * @version 1.2.4
 */
class ComplaintController extends Controller
{
    // ...
}
```

2.2 Method & Function DocBlocks

Every public, protected, and private method MUST include:
* A concise 1-line description of the method's purpose.
* `@param` annotations with explicit primitive or object types and parameter descriptions.
* `@return` annotation specifying the exact return type (e.g. `\Illuminate\Http\JsonResponse`, `bool`, `void`).
* `@throws` annotations if the method explicitly throws exceptions.

```
/**
 * Update the operational status of a complaint and record an immutable audit log.
 *
 * Validates the new status transition against permitted military workflows,
 * adjusts closure timestamps if resolved, and notifies the assigned employee.
 *
 * @param  \Illuminate\Http\Request  $request       The incoming HTTP request containing new status and remarks.
 * @param  \App\Models\Complaint    $complaint     The Eloquent complaint instance being transitioned.
 * @return \Illuminate\Http\RedirectResponse       Redirects back with a flash notification on success.
 * @throws \Illuminate\Validation\ValidationException When incoming request payloads fail business rules.
 */
public function updateStatus(Request $request, Complaint $complaint): RedirectResponse
{
    // Implementation...
}
```

2.3 Inline Logic Comments (The "Why", Not Just "What")

Do not write obvious comments like `// Increment i`. Write inline comments explaining **business rules, military procedures, edge cases, and security constraints**:

```
// STRICT PERMISSION CHECK: Admin role (ID 1 or role_name 'admin') has immutable full access
if ($this->id === 1 || $this->role_name === 'admin') {
    return true;
}

// Sub-module isolation: Strip action suffix (e.g. "houses.view" becomes "houses")
// to evaluate against exact module slugs in role_permissions table.
$module = explode('.', $permission)[0];

// Execute exact string comparison to prevent parent-to-child permission bubbling
return in_array($module, $permissions, true);
```

---

3. Real Codebase Examples & Audit Walkthroughs

3.1 Eloquent Model Standards: `app/Models/Role.php`

The `Role.php` model demonstrates proper relationship docblocks and model boot hook commenting:

```php
/**
 * Boot method to automatically assign all permissions to role_id = 1.
 *
 * Runs after role creation or update. If the role being saved is the master
 * Superadmin (ID = 1), automatically synchronizes all 19 module permissions
 * into role_permissions to prevent accidental lockout during releases.
 */
protected static function boot()
{
    parent::boot();

    static::saved(function ($role) {
        if ($role->id === 1) {
            $modules = [
                'dashboard', 'users', 'frontend-users', 'cmes', 'city', 'sector',
                'roles', 'employees', 'designation', 'houses', 'complaints',
                'category', 'sub-category', 'complaint-titles', 'approvals',
                'spares', 'reports', 'sla', 'registered-devices',
            ];

            // Inspect existing permissions to perform delta insert
            $existingModules = $role->rolePermissions()->pluck('module_name')->toArray();
            // ...
        }
    });
}
```

3.2 Authorization Middleware Standards: `app/Http/Middleware/CheckRolePermission.php`

Guarding routes with clear explanatory security comments:

```php
/**
 * Handle an incoming request and verify granular role permissions.
 *
 * @param  \Illuminate\Http\Request  $request
 * @param  \Closure  $next
 * @param  string  $permission  The module permission required (e.g. 'houses.view', 'cmes.view')
 * @return mixed
 */
public function handle(Request $request, Closure $next, string $permission)
{
    $user = auth()->user();

    // Deny immediately if unauthenticated
    if (!$user) {
        return redirect()->route('login');
    }

    // Evaluate exact-match permission via Role model
    if (!$user->hasPermission($permission)) {
        abort(403, 'Unauthorized access: You do not possess the required module permission: ' . $permission);
    }

    return $next($request);
}
```

---

4. Code Quality & Peer Review Checklist

Before any pull request or merge from `Amjad_branch` into `deploy`, the reviewing engineer MUST verify:

- [x] **Zero Raw SQL in Web Layer:** All queries use Eloquent ORM or parameterized query builder bindings.
- [x] **No Hardcoded Credentials:** All environment secrets, database passwords, and API keys are referenced via `env()` / `config()`.

- [x] **DocBlocks on All Functions:** Every newly added or modified method has complete `@param` and `@return` types.
- [x] **Status Constants Utilized:** Status values are referenced via `Complaint::STATUS_*` constants, never raw magic numbers (e.g. `2` or `0`).
- [x] **Rate Limit Enforced:** All new API endpoints have `throttle:` middleware applied.
- [x] **Soft Delete Compliance:** Models handling critical records implement `use SoftDeletes`.

© 2026 Pakistan Navy CMS • Nexertech Engineering